BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · 2026 · Lecturer: Mr. Jeewaka Perera

Faculty of Computing

Group Assignment

Search Algorithms in Pac-Man

> **Objective:** Implement the classic uninformed and informed search algorithms — DFS, BFS, Uniform Cost Search and A* — and design admissible, consistent heuristics for two structured problems (finding all corners, and eating all the food dots).

This is a Group assignment for a group of 4 members. Students must use the existing teams they have. Each group must complete the tasks together, and **everyone in the group must have a complete understanding of the entire solution**. You will be evaluated individually on your overall comprehension of the group's work, as well as your individual version control contributions.

Download link: search.zip.

Marking Scheme (Total: 100 Marks — 10% of Final Grade)

The total assignment is graded out of 100 marks, broken down into four components:

- **Autograder Score (40 Marks):** Group marks based on the autograder output for Q1-Q7.
- **Group Report (10 Marks):** Graded on structure, formatting, clarity, and completeness of the requirements.
- **Individual Git Contribution (10 Marks):** Assessed via report screenshots and repository verification.
- **Individual Viva Evaluation (40 Marks):** A rapid-fire individual assessment of your understanding of the code and algorithms. (Split into 20 marks for Algorithm Knowledge and 20 marks for Code Comprehension).

Autograder Allocation (40 Marks)

Question	Autograder	Marks
Q1 — Depth First Search	q1	4
Q2 — Breadth First Search	q2	4

Q3 — Uniform Cost Search (varying the cost function)	`q3`	4
Q4 — A* Search	`q4`	4
Q5 — Finding All the Corners (problem representation)	`q5`	8
Q6 — Corners Problem: Heuristic	`q6`	8
Q7 — Eating All The Dots: Food Heuristic	`q7`	8
Total		**40**

Part 1 — Project Setup

Section 1: Download & File Structure

1. Download `search.zip`, extract it, and open a terminal in the extracted `search` folder.

2. Confirm the folder contains `search.py`, `searchAgents.py`, `pacman.py`, `util.py` and `autograder.py`.

Project structure

File	Your role	Description
`search.py`	**Edit**	Where your search algorithms (DFS, BFS, UCS, A*) live — Q1–Q4.
`searchAgents.py`	**Edit**	Where search-based agents and problems are defined — Q5–Q7 (CornersProblem, heuristics).
`util.py`	Read only	Provides `Stack`, `Queue` and `PriorityQueue`. You must use these, not Python built-ins.
`pacman.py`	Reference	Runs the game; describes the `GameState` type.

`game.py`	Reference	Game logic — `AgentState`, `Agent`, `Directions`, `Grid`.
`autograder.py`	Run	Automated testing / grading script.
`test_cases/`	Auto-used	Contains the test cases for each question.

> **Critical:** Do **not** rename files, functions or classes. The autograder imports them by name; a rename means zero marks for that question.

Section 2: Environment & Verification

> **Requirements:** Python **3.9 – 3.11** | package manager Pip or Conda.

Create a new Conda environment and install the required packages:

```
conda create -n cs188 python=3.11
conda activate cs188
pip install numpy matplotlib
```

Verify the setup by playing a game of Pac-Man from inside the `search` folder:

```
python pacman.py
```

> **Success:** a Pac-Man game window opens and is playable with the arrow keys.

> **How search works here.** Every algorithm follows the same **graph-search** skeleton; only the fringe changes.
>
> 1. Put the start state on the fringe.
> 2. Pop a node from the fringe. If its state is the goal, return the path of actions that reached it.
> 3. If its state has not been expanded before, mark it expanded and push each successor (with the extended path, and accumulated cost where relevant) onto the fringe.
> 4. Repeat until the goal is found or the fringe is empty (return failure).

> DFS → `util.Stack` (LIFO) | BFS → `util.Queue` (FIFO) | UCS & A* → `util.PriorityQueue` (priority = *g* for UCS, *g* + *h* for A*).

Part 2 — Questions

Q1 — Finding a Fixed Food Dot using Depth First Search

4 marks · autograder q1

Implement the `depthFirstSearch` function in `search.py`. It must be the **graph-search** version: track expanded states so the same state is never expanded twice.

Autograder

```
python autograder.py -q q1
```

Q2 — Breadth First Search

4 marks · autograder q2

Implement `breadthFirstSearch` in `search.py`, again as graph search. The logic is almost identical to DFS — only the fringe changes.

Autograder

```
python autograder.py -q q2
```

Q3 — Uniform Cost Search (Varying the Cost Function)

4 marks · autograder q3

Implement `uniformCostSearch` in `search.py`. UCS generalises BFS by using the **actual accumulated path cost**, not the number of steps.

Autograder

```
python autograder.py -q q3
```

Q4 — A* Search

4 marks · autograder q4

Implement `aStarSearch` in `search.py`. It takes a `heuristic` argument — a function `heuristic(state, problem)` that estimates the remaining cost to a goal. The default is `nullHeuristic` (always 0), which makes A* degenerate to UCS.

Autograder

```
python autograder.py -q q4
```

Q5 — Finding All the Corners (Problem Representation)

8 marks · autograder q5

Implement the `CornersProblem` class in `searchAgents.py`. The goal is a path that touches **all four corners** of the maze. You must fill in `getStartState`, `isGoalState`, `getSuccessors` and successor bookkeeping.

Keep the state small and **hashable** so the expanded-set check works. Do not store the wall grid or the whole `GameState` in the state.

Autograder

```
python autograder.py -q q5
```

Q6 — Corners Problem: Heuristic

8 marks · autograder q6

Implement `cornersHeuristic` in `searchAgents.py` for the `CornersProblem` you built in Q5. It must be **admissible** and **consistent**, return **0** at any goal state, and never return a negative value.

Grading (nodes expanded on `mediumCorners`)

Nodes expanded	Marks
more than 2000	0 / 8

at most 2000	4 / 8
at most 1600	6 / 8
at most 1200	8 / 8

> **Inadmissible = 0 for the question.** If the autograder detects that A* with your heuristic returns a non-optimal path on any test, you score 0 regardless of node count.

Autograder

```
python autograder.py -q q6
```

Q7 — Eating All The Dots: Food Heuristic

8 marks · autograder q7

Implement `foodHeuristic` in `searchAgents.py` for the `FoodSearchProblem` (already provided). Your heuristic must be **admissible** and **consistent**.

Grading (nodes expanded on `trickySearch`)

Nodes expanded	Marks
more than 15000	2 / 8
at most 15000	4 / 8
at most 12000	6 / 8
at most 9000	8 / 8

Autograder

```
python autograder.py -q q7
```

Part 3 — Report, Git & Submission

Your group must submit a short PDF report alongside your code. The report contributes **10 marks** and must include the following for **each question (Q1-Q7)**:

- The specific functions/code blocks your group edited or added.
- A clear screenshot of the autograder output proving the score for that question.
- A concise explanation (**Max 200 words per question**) detailing the logic, data structures, and heuristic design used.

Additionally, the end of the report MUST contain:

1. **Public Git Repository Link:** The URL to your group's public Git repository used to collaborate on this assignment.
2. **Git Contribution Evidence:** Clear screenshots of your commit history and contribution graphs showing proper commit frequency, meaningful commit messages, and active contribution from *each* member throughout the assignment lifecycle.
3. **Individual Contribution Table:** Clearly listing the names, student IDs, and specific tasks/questions each member contributed to.
4. **AI Usage Declaration:** If any AI tools (e.g., ChatGPT, Gemini) were used for understanding the problem or generating ideas/code, explicitly state the tool used and include the exact prompts. If no AI was used, state "No AI tools were utilized."

Submission Format: You must submit TWO separate items to the course portal:

1. **Report:** Submit your `Report.pdf` as a standalone file. Name it after your Group ID (e.g., `Group_15_Report.pdf`).
2. **Code:** Put **ONLY** the two edited files (`search.py` and `searchAgents.py`) into a single ZIP file. Name the ZIP file after your Group ID (e.g., `Group_15_Code.zip`).

Only ONE submission of these two items is required per group.

Part 4 — Individual Git Contribution & Viva Evaluation (50 Marks)

Each student will be evaluated individually during a strict **6-minute Viva session per team**. This fast-paced session assesses your verifiable Git practices (10 Marks) and your immediate understanding of the group's code and core algorithms (40 Marks). **You are expected to understand the entire solution, not just the parts you wrote.**

Instructors will use the following quick-reference rubric to rapidly score students out of 50:

Criteria (Marks)	Excellent	Good	Above Average	Average	Below Average	
1. Git Contribution & Version Control (Max 10 Marks)	**[9 - 10]** Frequent, meaningful commits across timeline. Descriptive messages. Clear collaborative evidence.	**[7 - 8]** Regular commits, mostly clear messages. Minor clustering or gaps. Good work distribution.	**[6]** Somewhat infrequent commits. Vague messages. Shows basic usage but lacks consistency.	**[4 - 5]** Few commits, dumped in large chunks. Poor or meaningless messages.	**[2 - 3]** 1-2 commits total. No descriptive messages. Minimal evidence.	**[0** Git inc cl si
2. Viva: Core Algorithm Knowledge (Max 20 Marks)	**[18 - 20]** Flawless, instant explanation of DFS/BFS/UCS/A* concepts, fringe mechanics, and optimality.	**[15 - 17]** Confident explanation with minor hesitations. Solid grasp of core search mechanics.	**[12 - 14]** Good general understanding, but struggles slightly with A* nuances or optimality conditions.	**[8 - 11]** Can explain basic DFS/BFS but confused about UCS/A* or cost functions.	**[4 - 7]** Very vague. Cannot articulate how search algorithms operate beyond simple definitions.	Co ir an s (e c
3. Viva: Code Comprehension & Defense (Max 20 Marks)	**[18 - 20]** Deep mastery of code. Immediately points out how specific code handles states/heuristics. Mathematically defends admissibility.	**[15 - 17]** Solid understanding of the group's codebase. Defends heuristics well, minor prompting needed.	**[12 - 14]** Understands most code flow, but struggles to explain edge cases or mathematical proofs of heuristics.	**[8 - 11]** Only knows specific contributed portion. Struggles to explain overall state representation or other members' code.	**[4 - 7]** Guesses answers. Cannot map theoretical concepts to the actual Python implementation.	U i cor su

Total Individual Score: _____ / 50

FACULTY OF COMPUTING

```## Software Engineering Project & Viva Evaluation Rubric | Criteria | Excellent (80-100%) | Proficient (60-79%) | Basic (40-59%) | Needs Improvement (<40%) | Max Marks | Marks Awarded | | :--- | :--- | :--- | :--- | :--- | :--- | :--- | | **Implementation & Functionality** | All core and advanced requirements are fully implemented and execute flawlessly with edge cases handled. | Most core requirements function correctly; minor bugs present but do not break the main system flow. | Basic functionality works, but several core requirements are incomplete or crash frequently. | System fails to execute main requirements or requires significant debugging to run. | **30** | | | **Code Quality & Architecture** | Code is highly maintainable, consistently applying design principles (e.g., SOLID) and robust testing strategies (e.g., TDD). | Code is well-structured with acceptable modularity; some inconsistent application of design patterns or missing test coverage. | Code is monolithic or heavily coupled; basic logic is present but difficult to scale or maintain. | Code is highly disorganized, undocumented, and lacks logical architecture or separation of concerns. | **30** | | | **Viva Part 1: Technical Depth & Code Ownership** | Effortlessly explains underlying algorithms, justifies architectural choices, and successfully traces complex logic during questioning. | Understands the codebase well and correctly answers most technical questions, but struggles slightly with deeper architectural justification. | Shows surface-level understanding of the code; struggles to explain specific algorithms or relies heavily on partner contributions. | Unable to explain how key sections of the code work; strong indication of lack of contribution or understanding. | **20** | | | **Viva Part 2: Demonstration & Communication** | Delivers a clear, structured demonstration. Communicates technical concepts concisely and handles Q&A with high professional confidence. | Presents the system adequately. Communication is generally clear, though some responses to Q&A lack precision or structure. | Presentation is disjointed or rushed. Struggles to articulate technical concepts clearly to the panel. | Fails to successfully demonstrate the software. Communication is unclear, and Q&A responses are evasive or incorrect. | **20** | |```
```